



Sri Lanka Institute of Information Technology

Assignment 1

Report

Data Warehousing and Business Intelligence [IT3021]

Submitted by:

Registration Number	Name	Email
IT23227804	N.V.RANDUNUGE	it23227804@my.sliit.lk

Date of submission: 03/04/2026

Table of Contents

Introduction.....	3
Task 01: Dataset Description	4
Dataset Statistics	4
Entities Identified in the Database	5
Entity-Relationship Diagram (OLTP View).....	6
Task 02: Preparation of Data Sources.....	7
Task 03: Solution Architecture	8
Task 04: Data Warehouse Design & Development	9
The Grain Statement	9
Schema Type: Star Schema	10
Slowly Changing Dimension	10
Hierarchies for SSAS.....	11
Design Assumptions & Decisions	11
Implementation in SQL Server	12
Stagging	14
Task 05: ETL Development with SSIS.....	16
Task 6 : Accumulating Fact Table	19
Extending the Fact Table Schema.....	20
Package 3 : Data Flow Detail.....	21
Validation.....	22

Introduction

This report presents the design and implementation of a data warehouse solution based on a hotel booking reservation dataset. The project follows the standard DWBI lifecycle, including data source preparation, data warehouse design, and the development of Extract, Transform, Load (ETL) processes using SQL Server Integration Services (SSIS). The primary objective is to transform transactional booking data into a dimensional model that supports analytical querying and reporting.

The solution is built using a star schema design, incorporating fact and dimension tables to enable efficient data analysis. Key concepts such as Slowly Changing Dimensions (SCD), data staging, and accumulating fact tables are applied to ensure historical accuracy and support advanced business insights.

Overall, this project shows how data can be organized and processed to support better insights and decision-making.

Task 01: Dataset Description

Dataset – Hotel Booking Reservation

Data Source – [hotel-booking-reservation](#)

The selected dataset is the Hotel Bookings Dataset (2024), containing 119,390 hotel reservation records across 30 hotels in 15 Indian cities. Each row represents a single, real booking event, capturing everything from the guest's country of origin and room preferences to the average daily rate they paid and whether they ultimately cancelled.

The dataset spans the full calendar year of 2024 (January through December), with approximately 10,000 bookings per month distributed evenly across all months. Hotels are split between two types, City Hotels and Resort Hotels, spread across major Indian cities including Mumbai, Delhi, Hyderabad, Kolkata, Goa, and more.

Dataset Statistics

Metric	Value
Total Booking Records	119,390
Total Columns	33
Date Coverage	January 2024 – December 2024 (full year)
Number of Hotels	30 (15 City Hotels + 15 Resort Hotels)
Cities Covered	15 Indian cities
Unique Guest Countries	178 countries
Cancellation Rate	37.0% (44,224 out of 119,390)
Average Daily Rate (ADR)	INR 102 avg (range: -6 to 5,400)
Missing Values	children: 4 rows; country: 488; agent: 16,340; company: 112,593

Table 1 - Dataset Statistics

Entities Identified in the Database

Although the raw data is a flat CSV file, it actually contains information about multiple real-world entities mixed together. The entities are:

- **Hotel** : captured via the hotel column. Contains hotel name, type (City/Resort), and city. This is master reference data.
- **Booking** (the core transaction) : each row is one booking. Captured via dates, stay length, ADR, booking changes, deposit type, and status.
- **Guest / Customer** : captured via adults, children, babies, customer_type, country, is_repeated_guest. Represents who made the booking.
- **Room** : captured via reserved_room_type and assigned_room_type. Note guests often get a different room than they requested.
- **Meal Plan** : captured via the meal column. Represents what dining package was included.
- **Agent** : captured via the agent column. A numeric ID for the travel agent who made the booking (NULL = direct booking).
- **Market Segment** : captured via market_segment, distribution_channel, deposit_type. How and through what channel the booking was made.
- **Date** : captured via arrival_date_year, arrival_date_month, arrival_date_week_number, arrival_date_day_of_month.

Entity-Relationship Diagram (OLTP View)

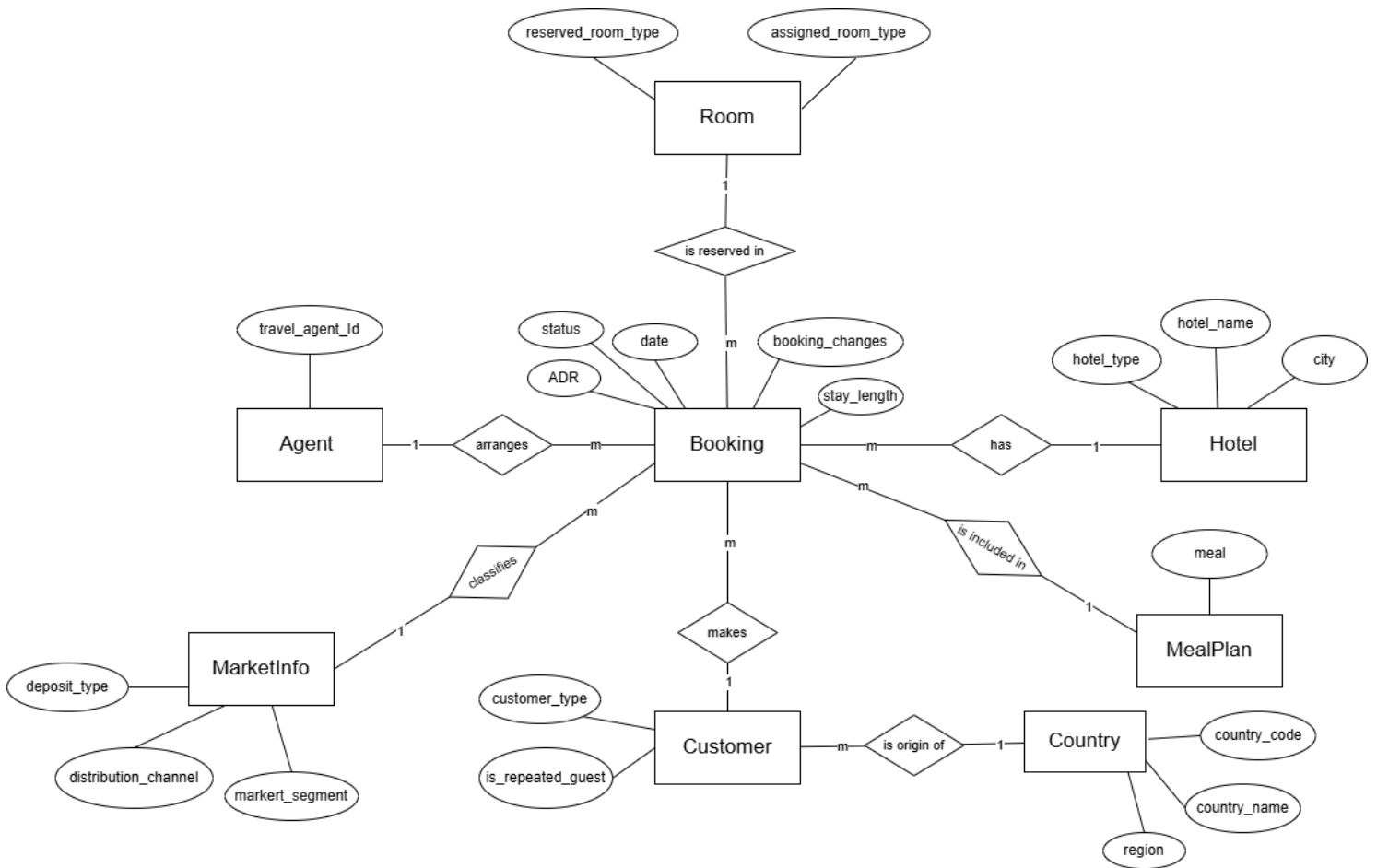


Figure 1 - Entity Relationship Diagram

Task 02: Preparation of Data Sources

Source	File / Object	Format	Contents	Feeds DW Object
Source 1	source1_hotels (SQL Table)	SQL Server Table	hotel_id, hotel_name, hotel_type, city 30 rows of hotel master data	dim_hotel
Source 2	source2_country_ref.xlsx	Excel (.xlsx)	country_code, country_name, region 177 country lookup rows	dim_country
Source 3	source3_bookings_core.csv	CSV Flat File	All 119,390 booking transactions with 33 columns, the main OLTP event log	fact_bookings + dim_customer, dim_room, dim_meal, dim_agent, dim_market, dim_date

Table 2 - Data Sources Description

Task 03: Solution Architecture

Architecture Overview

The solution follows the classic four-layer DW/BI reference architecture:

Source → Staging/ETL → Data Warehouse → BI

Each layer has a distinct responsibility, and data flows strictly from left to right (or top to bottom). This separation of concerns means that a problem in one layer (for example, a corrupt source file) can be isolated and fixed without affecting the other layers.

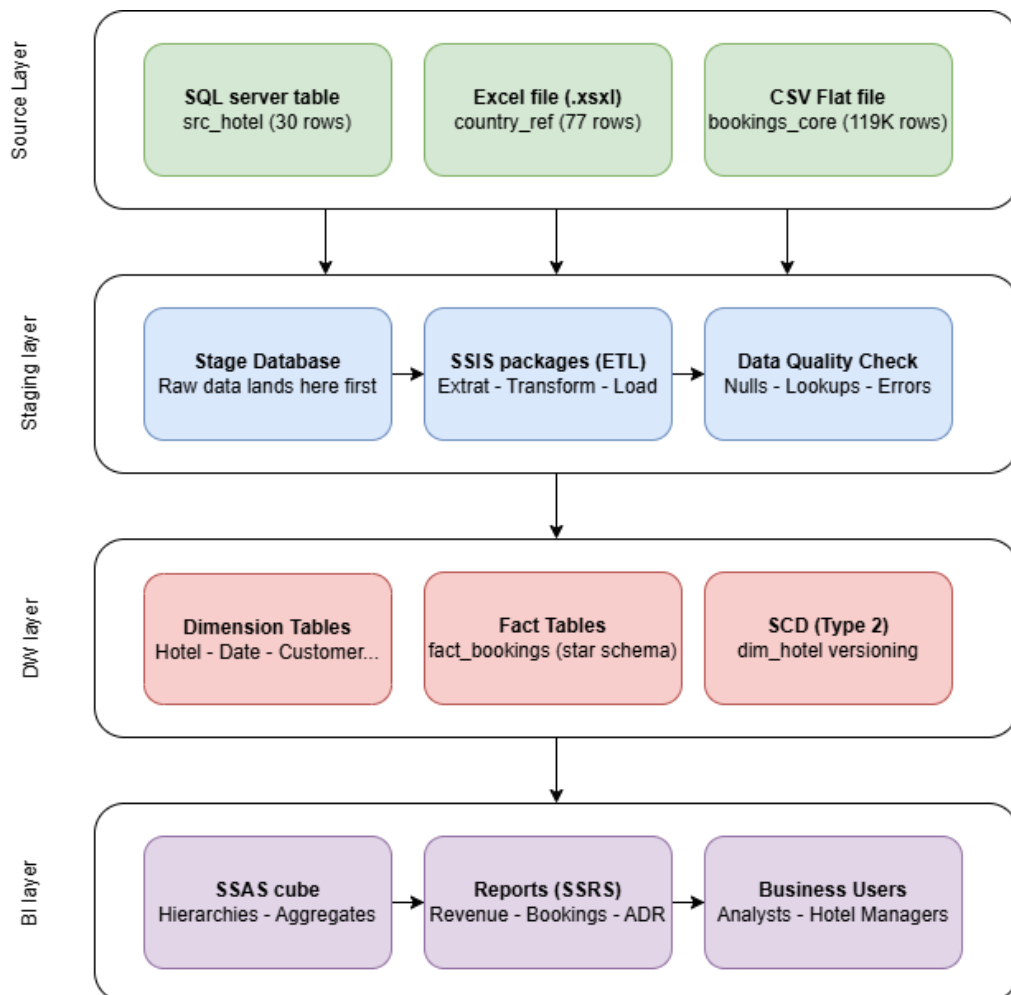


Figure 2 - Solution Architecture Diagram

Task 04: Data Warehouse Design & Development

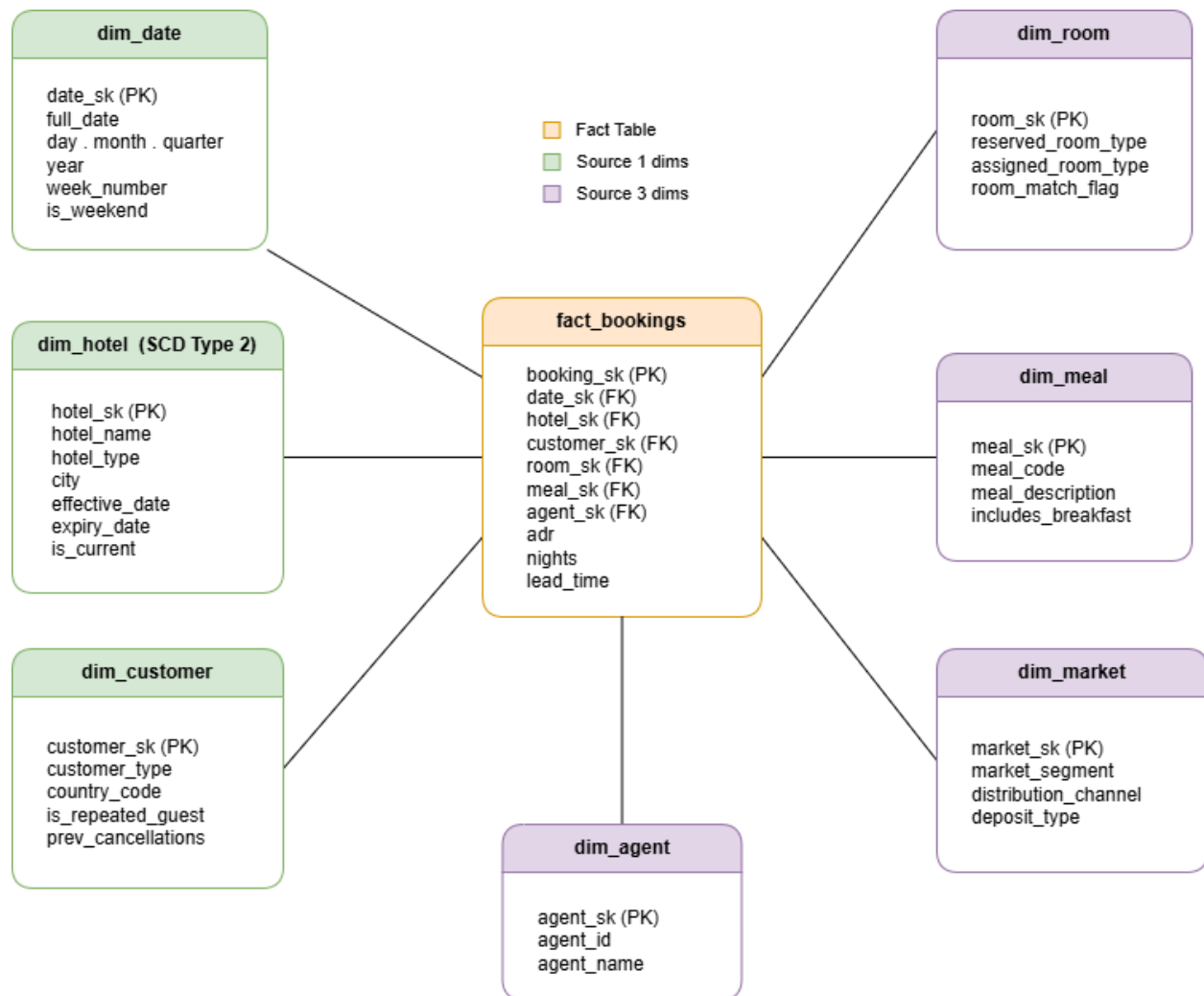


Figure 3 - Star Schema for Hotel Booking DW

The Grain Statement

The grain of this is that, each row in fact_bookings represents one hotel booking reservation -a single customer booking at a single hotel for a single arrival date.

This means, two bookings at the same hotel on the same day are two separate rows. One booking that was later cancelled is still one row (with is_canceled = 1). The grain never changes after it is defined.

Schema Type: Star Schema

This hotel bookings data warehouse design uses a Star Schema.

A Star Schema was chosen over a Snowflake Schema for its **better query performance**, as it uses simple joins between the fact table and dimension tables without complex chains. It is **also more compatible with SQL Server Analysis Services (SSAS)**, making cube creation easier through automatic relationship detection. Additionally, it **enhances readability**, allowing business users and developers to understand the schema without deep SQL knowledge.

Slowly Changing Dimension

In this warehouse, dim_hotel uses SCD Type 2 because hotel classifications are meaningful historically. If a hotel was reclassified from a City Hotel to a Resort Hotel in July 2024, bookings made in January 2024 should correctly reflect "City Hotel", not the new classification.

Consider a hotel starts as a City Hotel. In July it is reclassified as a Resort Hotel. Instead of overwriting the record, two rows now exist:

hotel_sk	hotel_id	hotel_name	hotel_type	city	eff_date	exp_date	is_current
1	1	City Hotel - Mumbai	City Hotel	Mumbai	2024-01-01	2024-06-30	0
2	1	City Hotel - Mumbai	Resort Hotel	Mumbai	2024-07-01	NULL	1

Old bookings (made before July) have hotel_sk = 1, so they correctly report as "City Hotel". New bookings (after July) have hotel_sk = 2 and correctly report as "Resort Hotel". The is_current = 1 flag tells SSIS which row to use when loading new bookings. The expiry_date = NULL on the current row means it has not yet been superseded.

Hierarchies for SSAS

Hierarchies that exist in this schema:

- Date hierarchy: Year → Quarter → Month → Day (all columns in dim_date)
- Hotel hierarchy: Hotel Type → City → Hotel Name (all columns in dim_hotel)
- Geography hierarchy: Region → Country → Country code
- Market hierarchy: Market Segment → Distribution Channel → Deposit Type (all in dim_market)

Design Assumptions & Decisions

- **Assumption 1- No individual guest identity:**

The dataset has no guest names, email addresses, or loyalty numbers. The dim_customer dimension therefore represents a customer profile (type + country + repeat status) rather than a named individual. Multiple bookings with the same profile share one customer_sk.

- **Assumption 2 - NULL agents = direct bookings:**

The 16,340 NULL values in the agent column are treated as direct/walk-in bookings. A dedicated dim_agent record (agent_id = 0, agent_name = "Direct / No Agent") is created so that the fact table foreign key is never NULL.

- **Assumption 3 - NULL children = 0 children:**

The 4 NULL values in the children column were replaced with 0 in the source preparation step (Task 2), as NULL children simply means no children — not missing data.

- **Assumption 4 - room_match_flag is derived:**

The column room_match_flag in dim_room is not in the source data — it is calculated during ETL as 1 if reserved_room_type = assigned_room_type, else 0.

Implementation in SQL Server

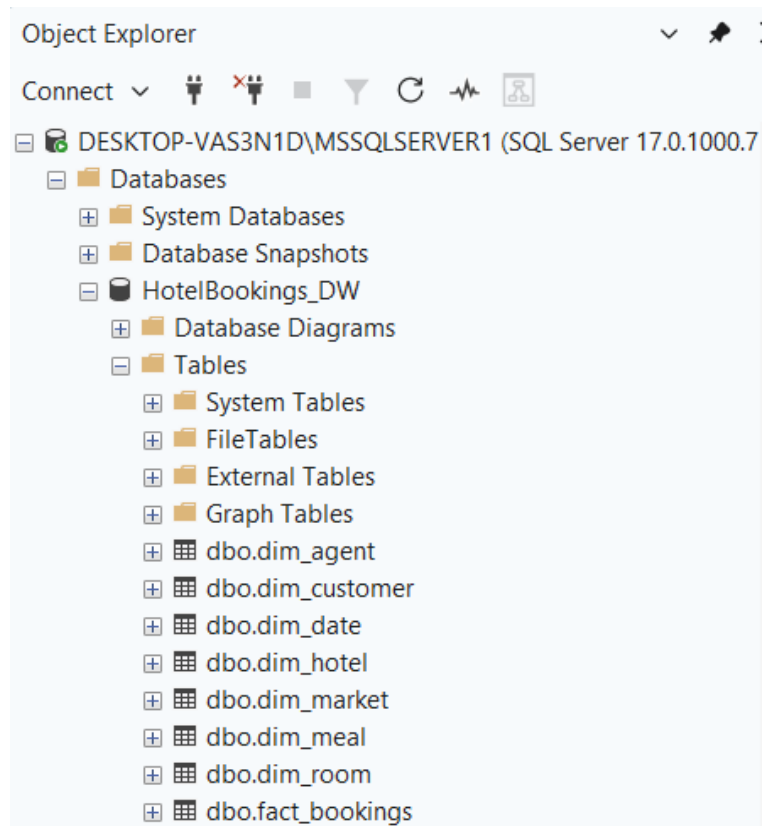


Figure 4 - Screenshot of Object Explorer in SSMS

Verification query to confirm row counts

```
-- -- VERIFICATION QUERIES --  
  
SELECT 'dim_date'      AS tbl, COUNT(*) AS rows FROM dbo.dim_date  
UNION ALL SELECT 'dim_hotel',      COUNT(*) FROM dbo.dim_hotel  
UNION ALL SELECT 'dim_customer',  COUNT(*) FROM dbo.dim_customer  
UNION ALL SELECT 'dim_room',      COUNT(*) FROM dbo.dim_room  
UNION ALL SELECT 'dim_meal',      COUNT(*) FROM dbo.dim_meal  
UNION ALL SELECT 'dim_market',    COUNT(*) FROM dbo.dim_market  
UNION ALL SELECT 'dim_agent',     COUNT(*) FROM dbo.dim_agent  
UNION ALL SELECT 'fact_bookings', COUNT(*) FROM dbo.fact_bookings;  
  
-- Expected before ETL:  
-- dim_date      = 366   (auto-populated above)  
-- dim_meal      = 5     (auto-populated above)  
-- dim_agent     = 1     (the 'no agent' seed record)  
-- all others    = 0     (will be filled by SSIS)  
-- fact_bookings = 0     (will be filled by SSIS)  
GO
```

Figure 5 – Screenshot of Verification Queries

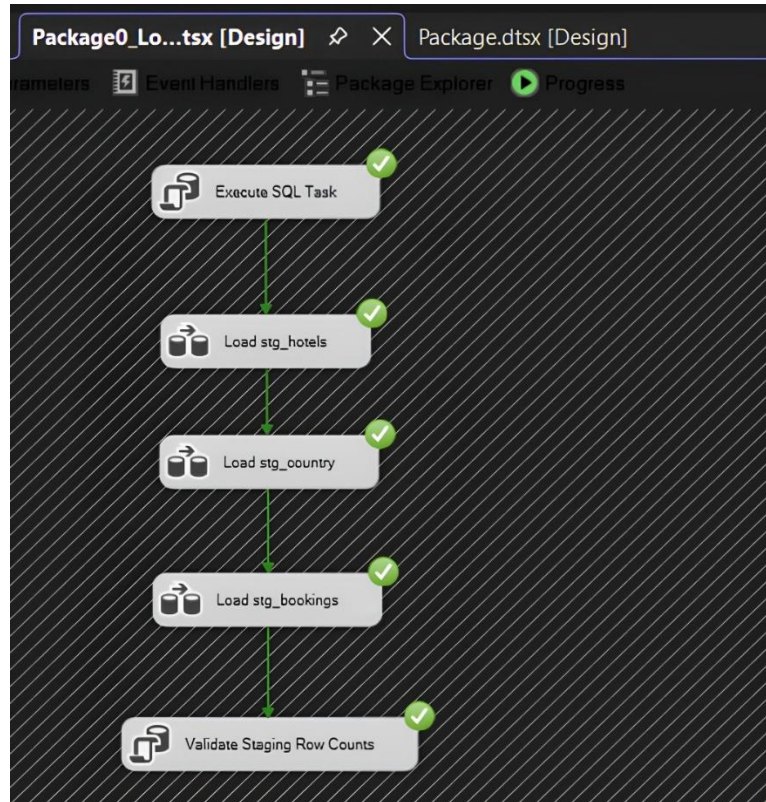
Results of the verification query:

	tbl	rows
1	dim_date	366
2	dim_hotel	0
3	dim_customer	0
4	dim_room	0
5	dim_meal	5
6	dim_market	0
7	dim_agent	1
8	fact_bookings	0

Figure 6 - Screenshot of the query result

Three tables are pre-populated: dim_date (366 rows for all of 2024), dim_meal (5 rows), and dim_agent (1 seed row for direct bookings).

Stagging



```
1  USE HotelBookings_Staging;
2
3  -- Row count check
4  SELECT 'stg_hotels' AS tbl, COUNT(*) AS rows FROM dbo.stg_hotels
5  UNION ALL
6  SELECT 'stg_country', COUNT(*) FROM dbo.stg_country
7  UNION ALL
8  SELECT 'stg_bookings', COUNT(*) FROM dbo.stg_bookings
9
10
11  SELECT TOP 3 * FROM dbo.stg_hotels;
12  SELECT TOP 3 * FROM dbo.stg_country;
13  SELECT TOP 3 hotel, arrival_date_month, adr, reservation_status
14  FROM   dbo.stg_bookings;
```

100 % No issues found

Results Messages

	tbl	rows
1	stg_hotels	30
2	stg_country	177
3	stg_bookings	119390

1

2

3

4

5

6

7

8

9

10

11

12

13

14

```
USE HotelBookings_Staging;

-- Row count check
SELECT 'stg_hotels' AS tbl, COUNT(*) AS rows FROM dbo.stg_hotels
UNION ALL
SELECT 'stg_country', COUNT(*) FROM dbo.stg_country
UNION ALL
SELECT 'stg_bookings', COUNT(*) FROM dbo.stg_bookings

SELECT TOP 3 * FROM dbo.stg_hotels;
SELECT TOP 3 * FROM dbo.stg_country;
SELECT TOP 3 hotel, arrival_date_month, adr, reservation_status
FROM dbo.stg_bookings;
```

100 %

No issues found

Results

Messages

	hotel_id	hotel_name	hotel_type	city	stg_load_dt
1	1	Resort Hotel - Chandigarh	Resort Hotel	Chandigarh	2026-03-30 12:16:34.413
2	2	Resort Hotel - Mumbai	Resort Hotel	Mumbai	2026-03-30 12:16:34.413
3	3	Resort Hotel - Delhi	Resort Hotel	Delhi	2026-03-30 12:16:34.413

	country_code	country_name	region	stg_load_dt
1	ABW	Country (ABW)	Other	2026-03-30 12:16:35.440
2	AGO	Angola	Africa	2026-03-30 12:16:35.440
3	AIA	Country (AIA)	Other	2026-03-30 12:16:35.440

	hotel	arrival_date_month	adr	reservation_status
1	Resort Hotel - Chandigarh	February	199.0	Canceled
2	Resort Hotel - Delhi	October	230.3	Canceled
3	Resort Hotel - Hyderabad	April	150.0	Canceled

Task 05: ETL Development with SSIS

The ETL is split into several packages :

Package	File name	Reads from	Writes to
Package 1	Package1_Load_Dimensions.dtsx	SQL table, Excel file, CSV	All 7 dimension tables
Package 2	Package2_Load_Fact.dtsx	Bookings CSV	fact_bookings
Package 3 (Task 6)	Package3_Accum_Fact.dtsx	Completions CSV	fact_bookings (UPDATE)

Package 1 : Load Dimensions

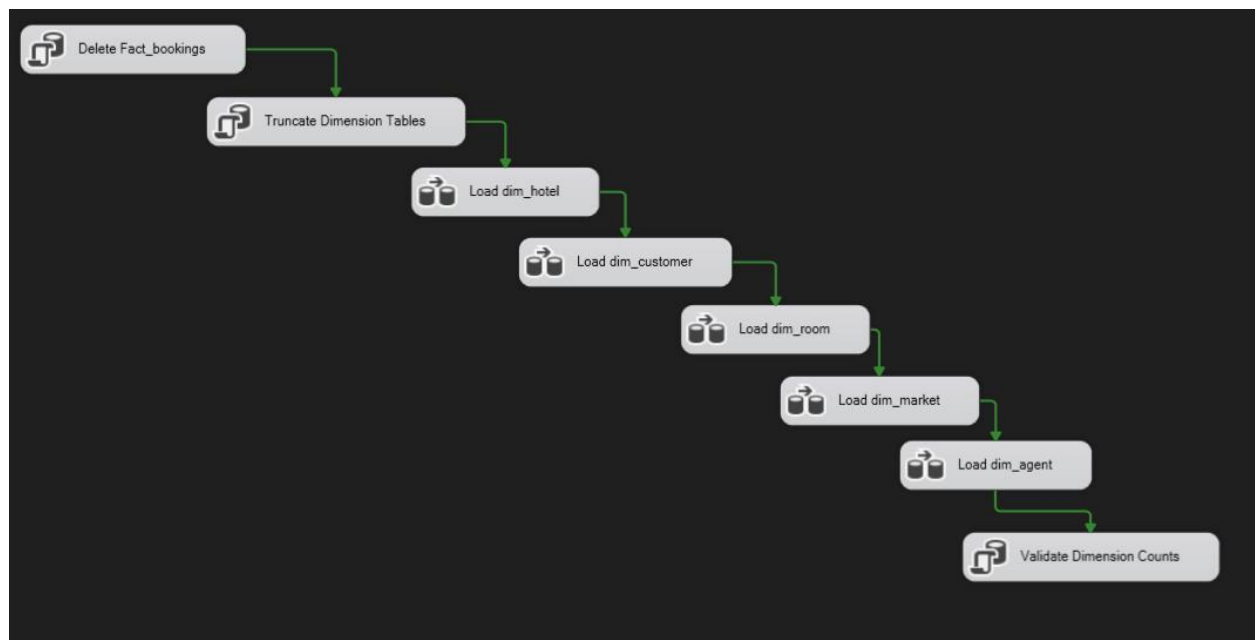


Figure 7 - Screenshot of SSIS Package1

Slowing changing dimensions

Slowly Changing Dimension Wizard

Slowly Changing Dimension Columns

Manage the changes to column data in your slowly changing dimensions by setting the



Fixed Attribute

Select this type when the value in a column should not change. Changes are treated as errors.

Changing Attribute

Select this type when changed values should overwrite existing values. This is a Type 1 change.

Historical Attribute

Select this type when changes in column values are saved in new

Select a change type for slowly changing dimension columns:

Dimension Columns	Change Type
city	Historical a...
hotel_name	Changing a...
hotel_type	Historical a...

Help

< Back

Next >

Finish >>|

Cancel

Package 2 : Load fact table

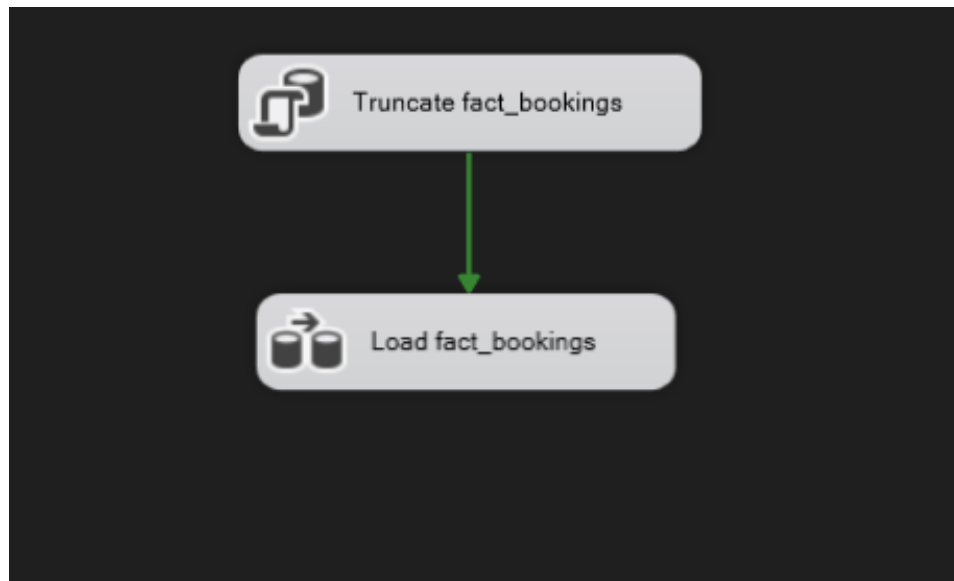


Figure 8 - Screenshot of SSIS Package 2



Figure 9 - Inside Load fact_bookings

Task 6 : Accumulating Fact Table

In this project, a hotel booking passes through two key milestones:

- Creation — the booking enters the data warehouse during the initial ETL load (Package 2).
- Completion — the booking is later marked as fully processed, with an associated completion timestamp.

The two additional columns added to fact_bookings to support this pattern are described in the table below.

Column Name	Data Type	Description
accm_txn_create_time	DATETIME	Records when the booking row was first inserted into the data warehouse fact table. Set to GETDATE() at load time by a Derived Column transform in Package 2.
accm_txn_complete_time	DATETIME	Records when the booking transaction was marked as completed. Initially NULL; updated by Package 3 when a completion record is received.
txn_process_time_hours	FLOAT	Computed as the difference in hours between accm_txn_complete_time and accm_txn_create_time. Calculated by Package 3 at the time of update.

Extending the Fact Table Schema

The fact_bookings table created in Task 4 is altered to add the three new columns. The SQL below was executed in SQL Server Management Studio (SSMS) against the HotelBookings_DW database

```
ALTER TABLE dbo.fact_bookings
ADD accm_txn_create_time DATETIME NULL,
    accm_txn_complete_time DATETIME NULL,
    txn_process_time_hours FLOAT NULL;
```

Setting accm_txn_create_time in Package 2

Package 2 (Package2_Load_Fact.dtsx) was updated to populate accm_txn_create_time at insert time. This was done by adding a Derived Column transform immediately before the OLE DB Destination inside the Package 2 Data Flow Task

Preparing the Completion Dataset

A separate source dataset was prepared to simulate the arrival of booking completion records. This file (source4_completions.csv) represents downstream system updates that confirm a booking transaction has been fully processed.

Sample File Content

```
txn_id,accm_txn_complete_time
1001,2024-08-17 14:23:00
1042,2024-08-18 09:45:30
2089,2024-08-20 16:10:15
3154,2024-08-21 11:00:00
5003,2024-08-22 08:30:45
```

Accumulating Fact Update Package

A new SSIS package named Package3_Accum_Fact.dtsx was created to read the completions CSV and update the corresponding rows in the fact_bookings table. This package implements the accumulating fact update pattern: it reads each completion record, matches it to a fact row using the natural key, and executes an SQL UPDATE to set accm_txn_complete_time and compute txn_process_time_hours.

Package 3 : Data Flow Detail

#	Component Name	Component Type	What it does
1	Flat File Source — completions CSV	Flat File Source	Reads source4_completions.csv row by row. Outputs two columns: txn_id (DT_I4) and accm_txn_complete_time (DT_STR).
2	Convert completion timestamp	Data Conversion	Converts the accm_txn_complete_time string column from DT_STR to DT_DBTIMESTAMP so it can be used in a datetime comparison and UPDATE. Output column named: completion_ts.
3	Compute process time hours	Derived Column	Adds a new column txn_process_time_hours_calc using the expression: (DT_R8)((completion_ts - accm_txn_create_time_from_fact) is not directly available here — instead the SQL UPDATE in the OLE DB Command calculates it inline). See note below.

4	OLE DB Command — update fact_bookings	OLE DB Command	Executes one parameterized SQL UPDATE statement per row, matching on booking_id = txn_id and setting both accm_txn_complete_time and txn_process_time_hours in a single statement.
---	--	-------------------	---

Validation

Validation Query 1 : Count of rows with completion time set

```
SELECT COUNT(*) AS completed_bookings
FROM    dbo.fact_bookings
WHERE   accm_txn_complete_time IS NOT NULL;
```

Result : equal the number of rows in source4_completions.csv that matched a booking_id in fact_bookings

Validation Query 2 : Spot-check - verify process time calculation

```
SELECT TOP 10
    booking_id,
    accm_txn_create_time,
    accm_txn_complete_time,
    txn_process_time_hours,
    DATEDIFF(SECOND, accm_txn_create_time,
accm_txn_complete_time) / 3600.0
        AS cross_check_hours
FROM    dbo.fact_bookings
WHERE   accm_txn_complete_time IS NOT NULL
ORDER  BY booking_id;
```

Result: identical txn_process_time_hours and cross_check_hours for every row, confirming the DATEDIFF calculation stored by Package 3 matches an independent recalculation

Validation Query 3 : Confirm NULL rows (bookings not yet completed)

```
SELECT COUNT(*) AS pending_bookings
FROM    dbo.fact_bookings
WHERE   accm_txn_complete_time IS NULL;
```

Result: completed_bookings + pending_bookings = 119,390 (total fact rows). All rows are accounted for

Validation Query 4 : Average processing time by hotel type

```
SELECT  d.hotel_type,
        COUNT(*)                AS completed_count,
        AVG(f.txn_process_time_hours) AS avg_process_hours,
        MIN(f.txn_process_time_hours) AS min_process_hours,
        MAX(f.txn_process_time_hours) AS max_process_hours
FROM    dbo.fact_bookings f
JOIN    dbo.dim_hotel d ON f.hotel_sk = d.hotel_sk
WHERE   f.accm_txn_complete_time IS NOT NULL
GROUP  BY d.hotel_type
ORDER  BY avg_process_hours DESC;
```

Demonstrates a real business insight enabled by the accumulating fact pattern. Shows the average processing time in hours per hotel type, allowing comparison of operational efficiency across hotel categories.

	rows_with_completion				
1	500				
	booking_id	accm_txn_create_time	accm_txn_complete_time	txn_process_time_hours	manual_check
1	1	2026-04-03 22:50:28.673	2026-04-07 13:50:28.673	87	87
2	2	2026-04-03 22:50:28.673	2026-04-04 23:50:28.673	25	25
3	3	2026-04-03 22:50:28.673	2026-04-06 15:50:28.673	65	65
4	4	2026-04-03 22:50:28.673	2026-04-08 16:50:28.673	114	114
5	5	2026-04-03 22:50:28.673	2026-04-09 12:50:28.673	134	134
6	6	2026-04-03 22:50:28.673	2026-04-09 20:50:28.673	142	142
7	7	2026-04-03 22:50:28.673	2026-04-05 08:50:28.673	34	34
8	8	2026-04-03 22:50:28.673	2026-04-05 21:50:28.673	47	47
9	9	2026-04-03 22:50:28.673	2026-04-08 02:50:28.673	100	100
10	10	2026-04-03 22:50:28.673	2026-04-09 21:50:28.673	143	143
	booking_id	accm_txn_create_time	accm_txn_complete_time	txn_process_time_hours	
1	1	2026-04-03 22:50:28.673	2026-04-07 13:50:28.673	87	